In [1]:
```
from dsc80_utils import *
```

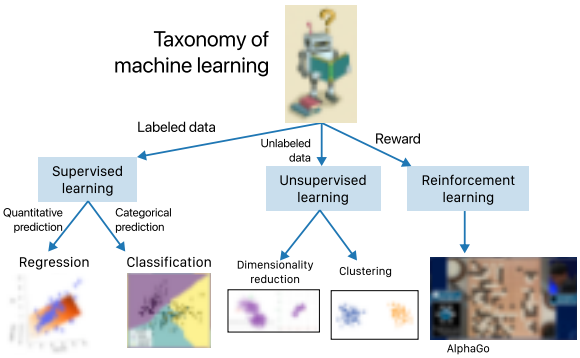
Lecture 17 – Random Forests, Classifier Evaluation

DSC 80, Fall 2025

Agenda

- Decision Trees
- Grid search
- Random forests
- Modeling with text features
- Classifier evaluation

Decision trees 🌲

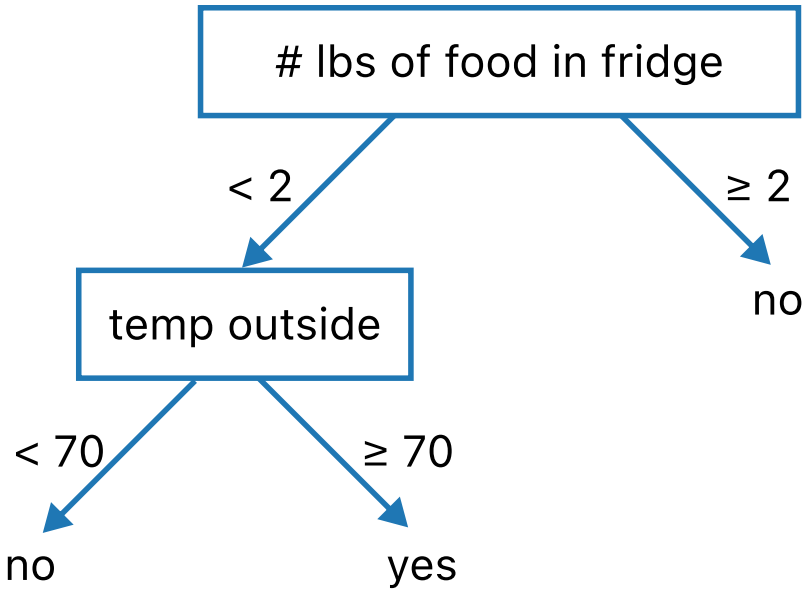


Decision trees can be used for both regression and classification. We'll start by using them for **classification**.

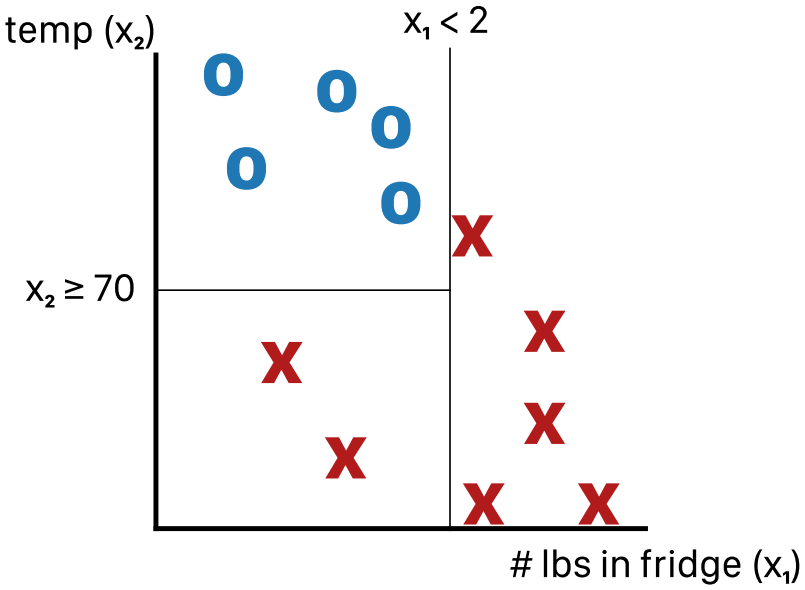
Example: Should I get groceries?

Decision trees make classifications by answering a series of yes/no questions.

Should I go to Trader Joe's to buy groceries today?



Internal **nodes** of trees involve questions; leaf nodes make predictions $H(x)$.



Example: Predicting diabetes

In [2]:
```
diabetes = pd.read_csv(Path('data') / 'diabetes.csv')
display_df(diabetes, cols=9)
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigr
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
...	...	...	...	...	...	...	
765	5	121	72	23	112	26.2	
766	1	126	60	0	0	30.1	
767	1	93	70	31	0	30.4	

768 rows × 9 columns

In [3]:
```
# 0 means no diabetes, 1 means yes diabetes.
diabetes['Outcome'].value_counts()
```

Out[3]:
```
Outcome
0    500
1    268
Name: count, dtype: int64
```

- `'Glucose'` is measured in mg/dL (milligrams per deciliter).
- `'BMI'` is calculated as $\text{BMI} = \frac{\text{weight (kg)}}{\left[\text{height (m)} \right]^2}$.
- Let's use `'Glucose'` and `'BMI'` to predict whether or not a patient has diabetes (`'Outcome'`).

Exploring the dataset

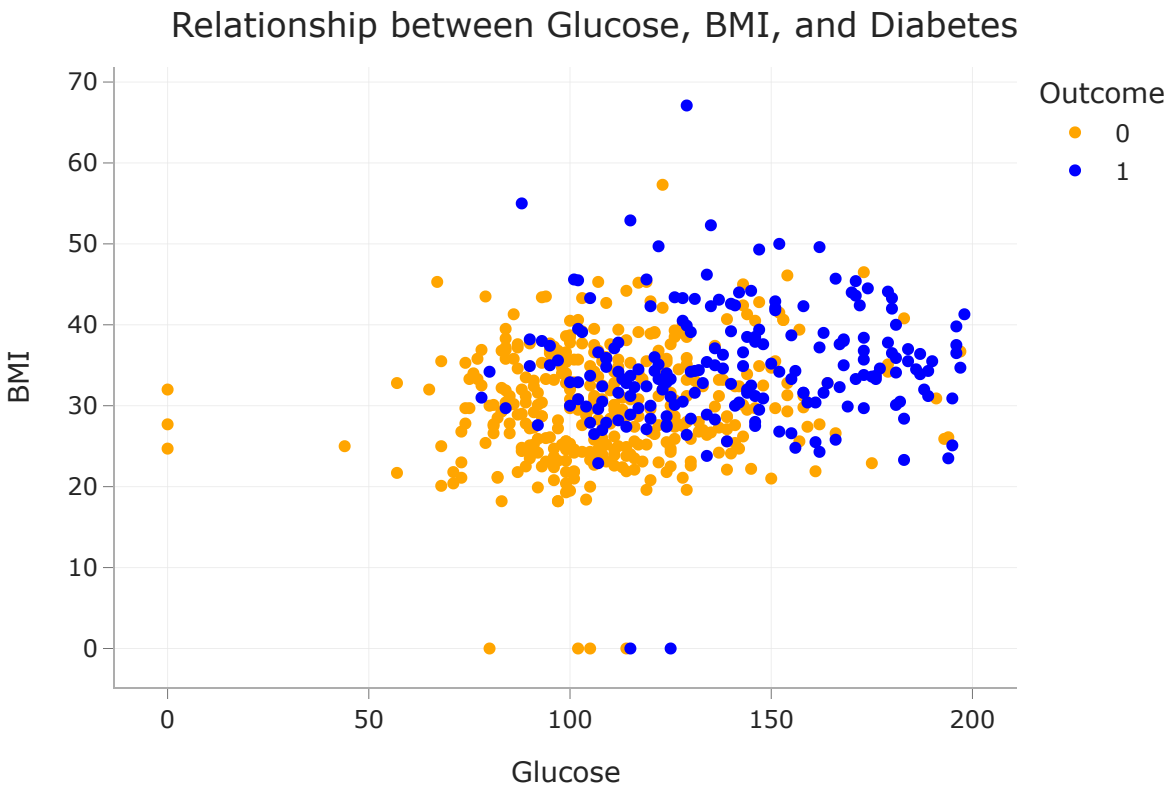
First, a train-test split:

In [4]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = (
    train_test_split(diabetes[['Glucose', 'BMI']], diabetes['Outcome'], ranc
)
```

Class 0 (orange) is "no diabetes" and **class 1 (blue) is "diabetes"**.

In [5]:
```
fig = (
    X_train.assign(Outcome=y_train.astype(str))
           .plot(kind='scatter', x='Glucose', y='BMI', color='Outcome',
                 color_discrete_map={'0': 'orange', '1': 'blue'},
                 title='Relationship between Glucose, BMI, and Diabetes')
```

```
)
fig
```



Building a decision tree

Let's build a decision tree and interpret the results.

The relevant class is `DecisionTreeClassifier`, from `sklearn.tree`.

In [6]:
```
from sklearn.tree import DecisionTreeClassifier
```

Note that we `fit` it the same way we `fit` earlier estimators.

You may wonder what `max_depth` and `criterion` do – more on this soon!

In [7]:
```
dt = DecisionTreeClassifier(max_depth=2, criterion='entropy')
```

In [8]:
```
dt.fit(X_train, y_train)
```

Out[8]:
```
DecisionTreeClassifier(criterion='entropy', max_depth=2)
```

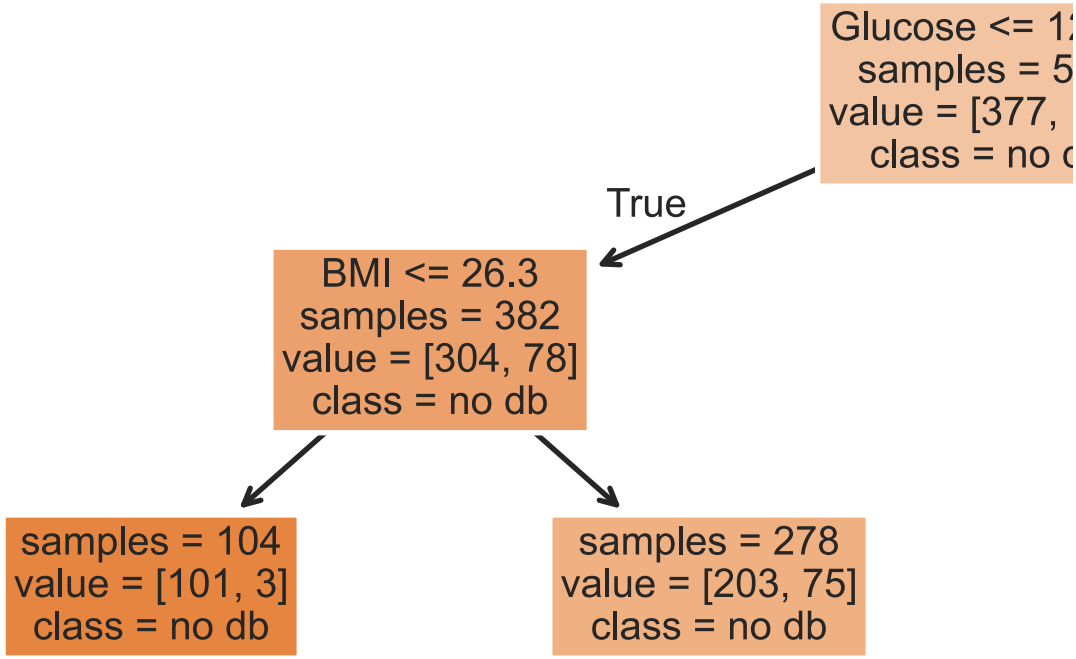
Visualizing decision trees

Our fit decision tree is like a "flowchart", made up of a series of questions.

As before, **orange is "no diabetes"** and **blue is "diabetes"**.

In [9]:
```
from sklearn.tree import plot_tree
```

In [10]:
```
plt.figure(figsize=(15, 5))
plot_tree(dt, feature_names=X_train.columns, class_names=['no db', 'yes db']
          filled=True, fontsize=15, impurity=False);
```



- To **classify a new data point**, we start at the top and answer the first question (i.e. "Glucose <= 129.5").
- If the answer is "**Yes**", we move to the **left** branch, otherwise we move to the right branch.
- We repeat this process until we end up at a leaf node, at which point we predict the most common class in that node.
  - Note that each node has a `value` attribute, which describes the number of **training** individuals of each class that fell in that node.

In [11]:
```
# Note that the left node at depth 2 has a `value` of [304, 78].
y_train[X_train.query('Glucose <= 129.5').index].value_counts()
```

Out[11]:
```
Outcome
0    304
1     78
Name: count, dtype: int64
```

Evaluating classifiers

The most common evaluation metric in classification is **accuracy**:

$$\text{accuracy} = \frac{\text{\# data points classified correctly}}{\text{\# data points}}$$

In [12]:
```
(dt.predict(X_train) == y_train).mean()
```

Out[12]:
```
np.float64(0.765625)
```

The `score` method of a classifier computes accuracy by default (just like the `score` method of a regressor computes R^2 by default). We want our classifiers to have **high accuracy**.

In [13]:
```
# Training accuracy — same number as above
dt.score(X_train, y_train)
```

Out[13]:
```
0.765625
```

In [14]:
```
# Testing accuracy
dt.score(X_test, y_test)
```

Out[14]:
```
0.7760416666666666
```

Reflection

- Decision trees are easily interpretable: it is clear *how* they make their predictions.
- They work with categorical data without needing to use one hot encoding.
- They also can be used in multi-class classification problems, e.g. when there are more than 2 possible outcomes.
- The *decision boundary* of a decision tree can be arbitrarily complicated.
- **How are decision trees trained?**

How are decision trees trained?

Pseudocode:

```
def make_tree(X, y):
    if all points in y have the same label C:
        return Leaf(C)
```

```
    f = best splitting feature # e.g. Glucose or BMI
    v = best splitting value   # e.g. 129.5

    X_left, y_left   = X, y where (X[f] <= v)
    X_right, y_right = X, y where (X[f] > v)

    left  = make_tree(X_left, y_left)
    right = make_tree(X_right, y_right)

    return Node(f, v, left, right)

make_tree(X_train, y_train)
```

How do we measure the quality of a split?

Our pseudocode for training a decision tree relies on finding the best way to "split" a node – that is, **the best question to ask** to help us narrow down which class to predict.

Intuition: Suppose the distribution within a node looks like this (colors represent classes):

🟠🟠🟠🟠🟠🟠🔵🔵🔵🔵🔵🔵🔵

Split A:

- "Yes": 🟠🟠🟠🔵🔵🔵
- "No": 🟠🟠🟠🔵🔵🔵🔵

Split B:

- "Yes": 🔵🔵🔵🔵🔵🔵
- "No": 🔵🟠🟠🟠🟠🟠🟠

Which split is "better"?

Split B, because there is "less uncertainty" in the resulting nodes in Split B than there is in Split A. Let's try and quantify this!

Entropy

- For each class C within a node, define p_C as the proportion of points with that class.
  - For example, the two classes may be "yes diabetes" and "no diabetes".

- The **surprise** of drawing a point from the node at random and having it be class C is:

$$ - \log_2 p_C $$

- The **entropy** of a node is the average surprise over all classes:

\begin{align} \text{entropy} &= - \sum_C p_C \log_2 p_C \end{align}

- The entropy of 🟠🟠🟠🟠🟠🟠🟠🟠 is $ -1 \log_2(1) = 0 $.

- The entropy of 🟠🟠🟠🟠🔵🔵🔵🔵 is $ -0.5 \log_2(0.5) - 0.5 \log_2(0.5) = 1$.

- The entropy of 🟠🔵🟢🟡🟣 is $ - \log_2 \frac{1}{5} = \log_2(5) $
  - In general, if a node has n points, all with different labels, the entropy of the node is $ \log_2(n) $.

Example entropy calculation

Suppose we have:

🟠🟠🟠🟠🟠🟠🟠🟠🟠🟠🟠🟠🔵🔵🔵🔵🔵🔵

Split A:

- "Yes": 🟠🟠🟠🟠🟠🟠🔵
- "No": 🟠🟠🟠🟠🟠🟠🔵🔵🔵🔵🔵

Split B:

- "Yes": 🟠🟠🟠🟠🟠🟠🔵🔵🔵
- "No": 🟠🟠🟠🟠🟠🟠🔵🔵🔵

We choose the split that has the lowest **weighted entropy**, that is:

$$\text{entropy of split} = \frac{\# \text{Yes}}{\# \text{Yes} + \# \text{No}} \cdot \text{entropy(Yes)} + \frac{\# \text{No}}{\# \text{Yes} + \# \text{No}} \cdot \text{entropy(No)}$$

In [15]:
```
def entropy(node):
    props = pd.Series(list(node)).value_counts(normalize=True)
    return -sum(props * np.log2(props))
```

In [16]:
```
def weighted_entropy(yes_node, no_node):
    yes_entropy = entropy(yes_node)
    no_entropy = entropy(no_node)
    yes_weight = len(yes_node) / (len(yes_node) + len(no_node))
    return yes_weight * yes_entropy + (1 - yes_weight) * no_entropy
```

In [17]:
```
# Split A:
weighted_entropy("🟠🟠🟠🟠🟠🟠🔵", "🟠🟠🟠🟠🟠🟠🔵🔵🔵🔵🔵")
```

Out[17]: 0.8375578764623786

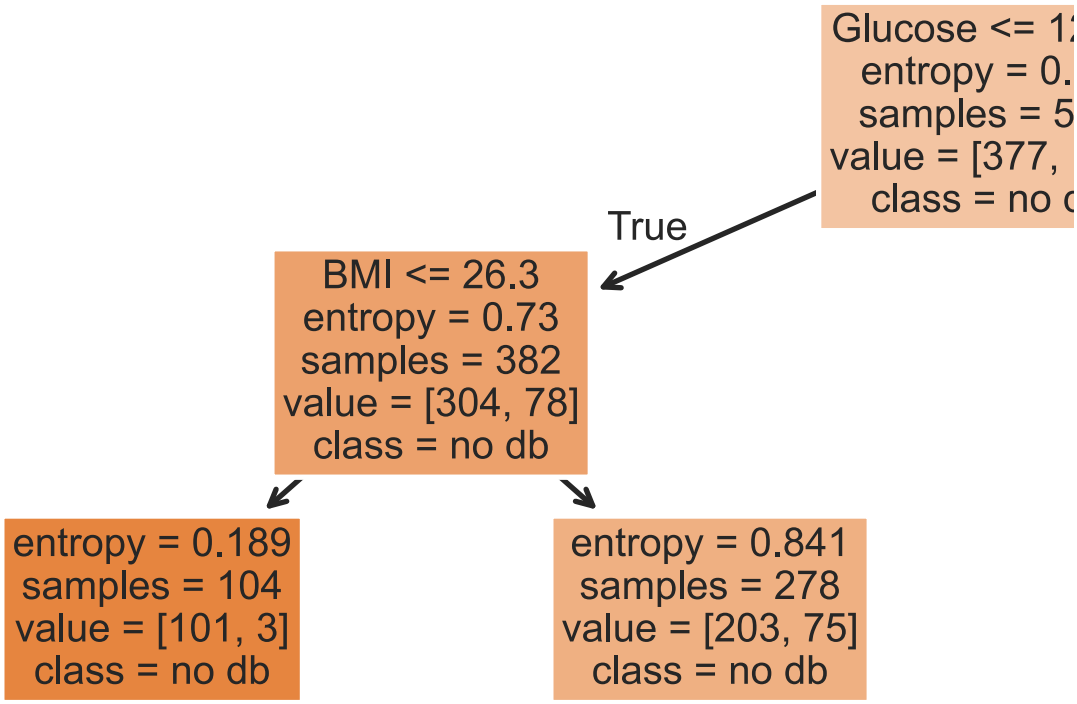
In [18]:
```
# Split B:
weighted_entropy("🟠🟠🟠🟠🟠🟠🔵🔵🔵", "🟠🟠🟠🟠🟠🟠🔵🔵🔵")
```

Out[18]: 0.9182958340544896

Split A has the lower weighted entropy, so we'll use it.

Understanding entropy

In [19]:
```
plt.figure(figsize=(15, 5))
plot_tree(dt, feature_names=X_train.columns, class_names=['no db', 'yes db']
          filled=True, fontsize=15, impurity=True);
```



We can recreate the entropy calculations above! Note that the default `DecisionTreeClassifier` uncertaintly metric *isn't* entropy; it used entropy because we set `criterion='entropy'` when defining `dt`. (The default metric, Gini impurity, ine too!)

In [20]:
```
# The first node at depth 2 has an entropy of 0.73,
# both told to us above and verified here!
entropy([0] * 304 + [1] * 78)
```

Out[20]:
```
0.7302263747422792
```

Tree depth

Decision trees are trained by **recursively** picking the best split until:

- all "leaf nodes" only contain training examples from a single class (good), or
- it is impossible to split leaf nodes any further (not good).

By default, there is no "maximum depth" for a decision tree. As such, without restriction, decision trees tend to be very deep.

In [21]:
```
dt_no_max = DecisionTreeClassifier()
dt_no_max.fit(X_train, y_train)
```

Out[21]:
```
DecisionTreeClassifier()
```

A decision tree fit on our training data has a depth of around 20! (It is so deep that `tree.plot_tree` errors when trying to plot it.)

In [22]:
```
dt_no_max.tree_.max_depth
```

Out[22]:
```
20
```

At first, this tree seems "better" than our tree of depth 2, since its training accuracy is much much higher:

In [23]:
```
dt_no_max.score(X_train, y_train)
```

Out[23]:
```
0.9913194444444444
```

In [24]:
```
# Depth 2 tree.
dt.score(X_train, y_train)
```

Out[24]:
```
0.765625
```

But recall, we truly care about **test set performance**, and this decision tree has **worse accuracy on the test set than our depth 2 tree**.

In [25]:
```
dt_no_max.score(X_test, y_test)
```

In [26]:
```
# Depth 2 tree.
dt.score(X_test, y_test)
```

Out[26]: 0.7760416666666666

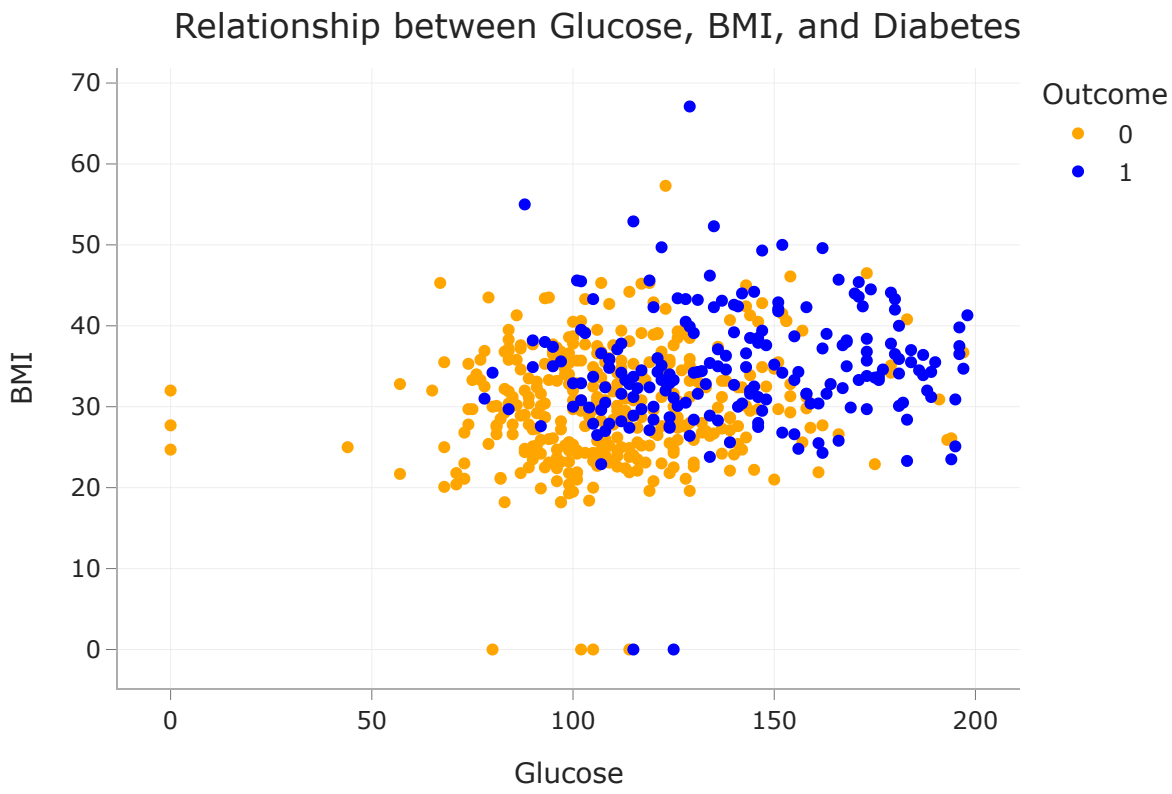
Decision trees and overfitting

- Decision trees have a tendency to overfit. **Why is that?**

- Unlike linear classification techniques (like logistic regression), **decision trees are non-linear**.
  - They are also "non-parametric" – there are no w^*s to learn.

- While being trained, decision trees ask enough questions to effectively **memorize** the correct response values in the training set. However, the relationships they learn are often overfit to the noise in the training set, and don't generalize well.

In [27]:
```
fig
```



- A decision tree whose depth is not restricted will achieve 100% accuracy on any training set, as long as there are no "overlapping values" in the training set.
  - Two values overlap when they have the same features x but different response values y (e.g. if two patients have the same glucose levels and BMI,

but one has diabetes and one doesn't).

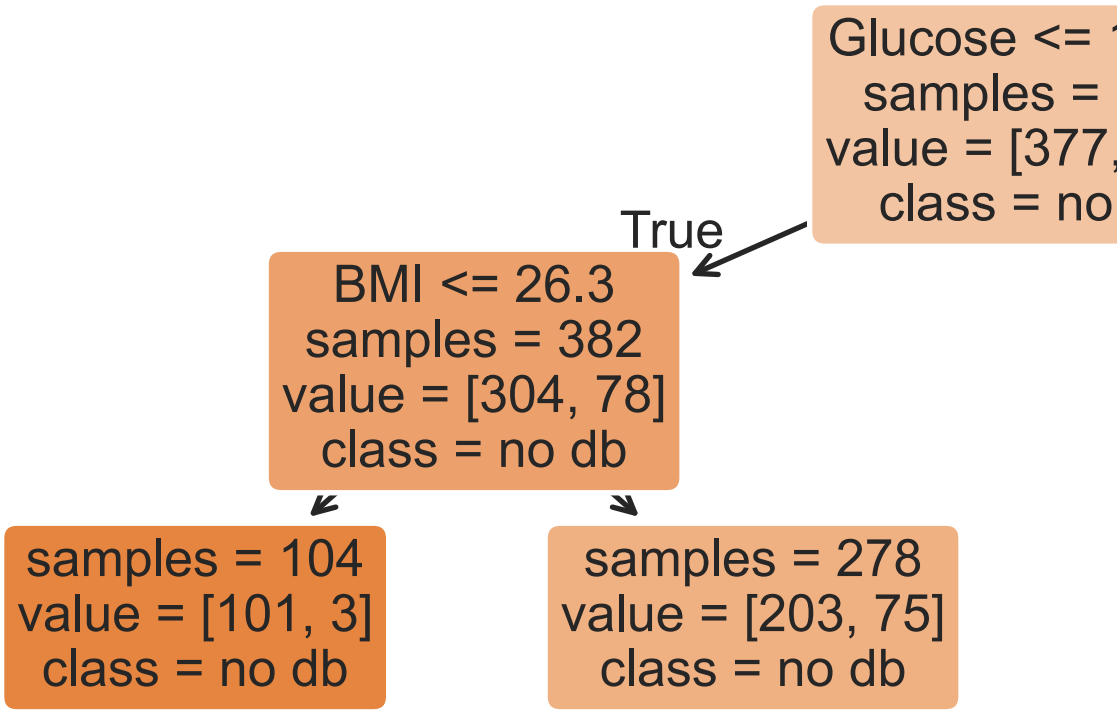
- **One solution**: Make the decision tree "less complex" by limiting the maximum depth.

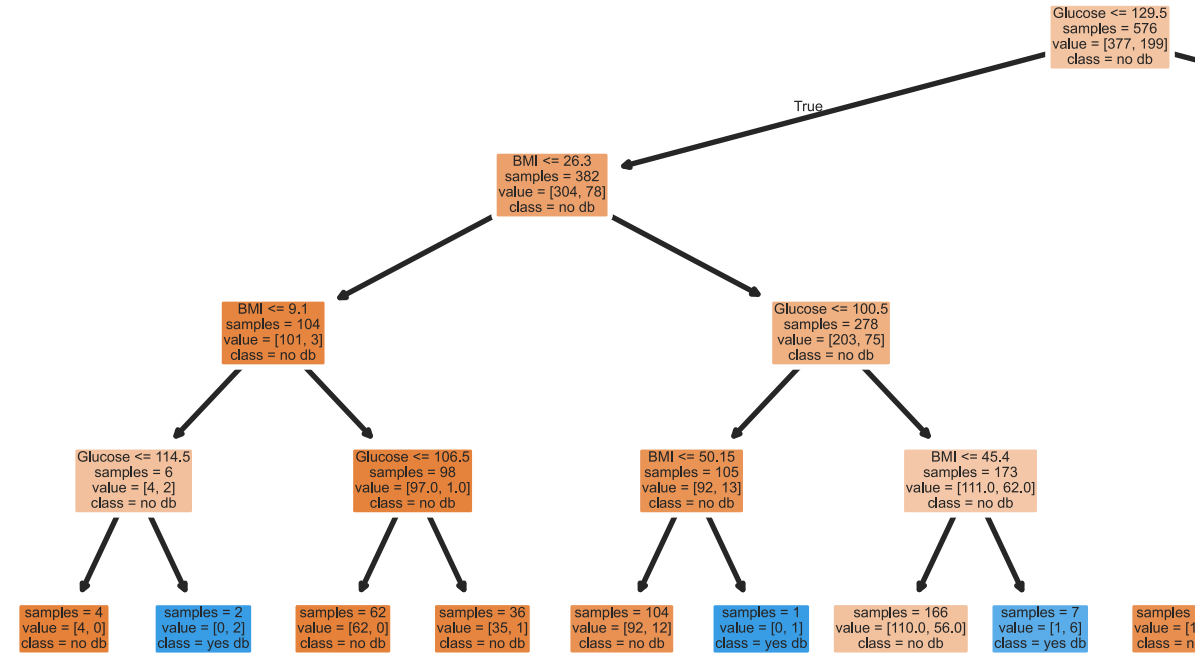
Since `sklearn.tree`'s `plot_tree` can't visualize extremely large decision trees, let's create and visualize some smaller decision trees.

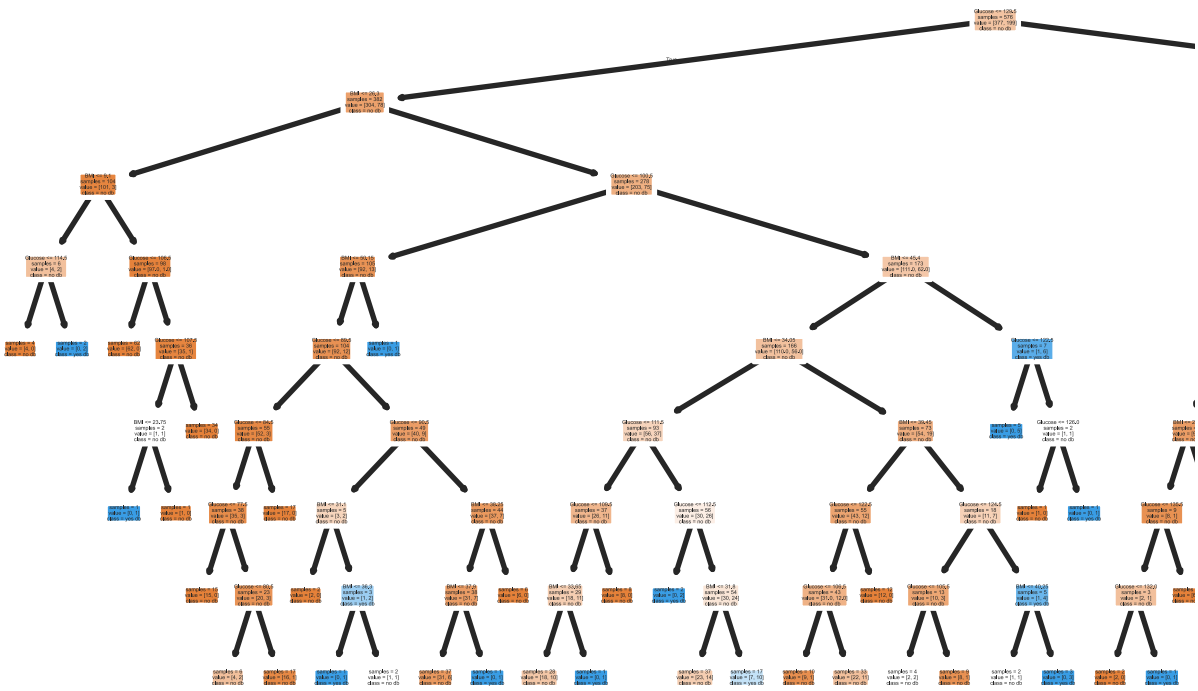
In [28]:
```
trees = {}
for d in [2, 4, 8]:
    trees[d] = DecisionTreeClassifier(max_depth=d, random_state=1)
    trees[d].fit(X_train, y_train)

    plt.figure(figsize=(15, 5), dpi=100)
    plot_tree(trees[d], feature_names=X_train.columns, class_names=['no db',
              filled=True, rounded=True, impurity=False)

    plt.show()
```







As tree depth increases, complexity increases, and our trees are more prone to overfitting. This means model bias decreases, but model variance increases.

Question: What is the "right" maximum depth to choose?

Hyperparameters for decision trees

- `max_depth` is a hyperparameter for `DecisionTreeClassifier`.

- There are many more hyperparameters we can tweak; look at the documentation for examples.
  - `min_samples_split`: The minimum number of samples required to split an internal node.
  - `criterion`: The function to measure the quality of a split (`'gini'` or `'entropy'`).

- To ensure that our model generalizes well to unseen data, we need an efficient technique for trying different combinations of hyperparameters!

- We'll see that technique soon: `GridSearchCV`.

Thinking about bias and variance

- Bigger `max_depth` = less bias, more variance.
- Bigger `min_samples_split` = more bias, less variance. (Why?)

Grid search

Grid search

`GridSearchCV` takes in:

- an **un-`fit`** instance of an estimator, and
- a **dictionary** of hyperparameter values to try,

and performs k-fold cross-validation to find the **combination of hyperparameters with the best average validation performance**.

In [29]:
```
from sklearn.model_selection import GridSearchCV
```

The following dictionary contains the values we're considering for each hyperparameter. (We're using `GridSearchCV` with 3 hyperparameters, but we could use it with even just a single hyperparameter.)

In [30]:
```
hyperparameters = {
    'max_depth': [2, 3, 4, 5, 7, 10, 13, 15, 18, None],
    'min_samples_split': [2, 5, 10, 20, 50, 100, 200],
    'criterion': ['gini', 'entropy']
}
```

Note that there are 140 **combinations** of hyperparameters we need to try. We need to find the **best combination** of hyperparameters, not the best value for each hyperparameter individually.

In [31]:
```
len(hyperparameters['max_depth']) * \
len(hyperparameters['min_samples_split']) * \
len(hyperparameters['criterion'])
```

Out[31]:
```
140
```

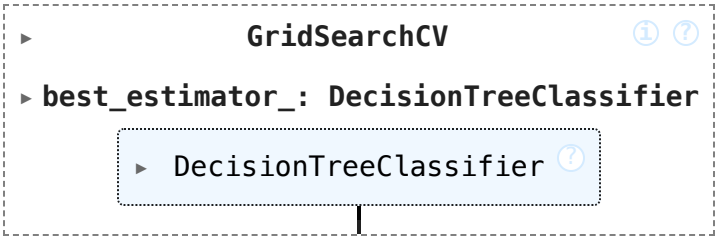
`GridSearchCV` needs to be instantiated and `fit`.

In [32]:
```
searcher = GridSearchCV(DecisionTreeClassifier(), hyperparameters, cv=5)
```

In [33]:
```
%%time
searcher.fit(X_train, y_train)
```

```
CPU times: user 916 ms, sys: 9.86 ms, total: 926 ms
Wall time: 984 ms
```

Out[33]:



After being `fit`, the `best_params_` attribute provides us with the best combination of hyperparameters to use.

In [34]:
```
searcher.best_params_
```

Out[34]:
```
{'criterion': 'entropy', 'max_depth': 5, 'min_samples_split': 50}
```

All of the intermediate results – validation accuracies for each fold, mean validation accuaries, etc. – are stored in the `cv_results_` attribute:

In [35]:
```
searcher.cv_results_['mean_test_score'] # Array of length 140.
```

Out[35]:
```
array([0.73, 0.73, 0.73, ..., 0.75, 0.74, 0.72])
```

In [36]:
```
# Rows correspond to folds, columns correspond to hyperparameter combinatior
pd.DataFrame(np.vstack([searcher.cv_results_[f'split{i}_test_score'] for i i
```

Out[36]:

	0	1	2	3	...	136	137	138	139
0	0.71	0.71	0.71	0.71	...	0.67	0.70	0.71	0.73
1	0.77	0.77	0.77	0.77	...	0.82	0.83	0.77	0.76
2	0.74	0.74	0.74	0.74	...	0.68	0.72	0.74	0.73
3	0.70	0.70	0.70	0.70	...	0.77	0.79	0.76	0.70
4	0.72	0.72	0.72	0.72	...	0.70	0.71	0.72	0.70

5 rows × 140 columns

Note that the above DataFrame tells us that 5 * 140 = 700 models were trained in total!

Now that we've found the best combination of hyperparameters, we should fit a decision tree instance using those hyperparameters on our entire training set.

In [37]:
```
searcher.best_params_
```

Out[37]:
```
{'criterion': 'entropy', 'max_depth': 5, 'min_samples_split': 50}
```

In [38]:
```
final_tree = DecisionTreeClassifier(**searcher.best_params_)
final_tree
```

Out[38]:
```
DecisionTreeClassifier(criterion='entropy', max_depth=5, min_samples_split=50)
```

In [39]:
```
final_tree.fit(X_train, y_train)
```

Out[39]:
```
DecisionTreeClassifier(criterion='entropy', max_depth=5, min_samples_split=50)
```

In [40]:
```
# Training accuracy.
final_tree.score(X_train, y_train)
```

Out[40]:
```
0.7829861111111112
```

In [41]:
```
# Testing accuracy.
# A bit lower than the `dt` tree we fit above!
final_tree.score(X_test, y_test)
```

Out[41]:
```
0.765625
```

Remember, `searcher` itself is a model object (we had to `fit` it). After performing k-fold cross-validation, behind the scenes, `searcher` is trained on the entire training set using the optimal combination of hyperparameters.

In other words, `searcher` makes the same predictions that `final_tree` does!

In [42]:
```
searcher.score(X_train, y_train)
```

Out[42]:
```
0.7829861111111112
```

In [43]:
```
searcher.score(X_test, y_test)
```

Out[43]:
```
0.765625
```

Choosing possible hyperparameter values

- A full grid search can take a **long time**.
  - In our previous example, we tried 140 combinations of hyperparameters.
  - Since we performed 5-fold cross-validation, we trained 700 decision trees under the hood.
- **Question**: How do we pick the possible hyperparameter values to try?
- **Answer**: Trial and error.
  - If the "best" choice of a hyperparameter was at an extreme, try increasing the range.
  - For instance, if you try `max_depth`s from 32 to 128, and 32 was the best, try including `max_depths` under 32.

Key takeaways

- Decision trees are trained by finding the best questions to ask using the features in the training data. A good question is one that isolates classes as much as possible.
- Decision trees have a tendency to overfit to training data. One way to mitigate this is by restricting the maximum depth of the tree.
- To efficiently find hyperparameters through cross-validation, use `GridSearchCV`.
  - Specify which values to try for each hyperparameter, and `GridSearchCV` will try all **unique combinations of hyperparameters** and return the combination with the best average validation performance.
  - `GridSearchCV` is not the only solution – see `RandomizedSearchCV` if you're curious.

Decision tree pros and cons

Pros:

- Fast to fit (usually log-linear time w.r.t. size of design matrix)
- Fast to predict (usually log time)
- Interpretable
- Robust to irrelevant features (think about why!)
- Linear transformations on features don't affect predictions (think about why!)

Cons:

- High variance: complete tree (no depth limit) will almost always overfit!
- Aren't the best at prediction in general.

Random Forests

Another idea:

Train a bunch of decision trees, then have them **vote** on a prediction!

- **Problem:** If you use the same training data, you will always get the same tree.
- **Solution:** Introduce randomness into training procedure to get different trees.

Idea 1: Bootstrap the training data

- We can bootstrap the training data T times, then train one tree on each resample.
- Also known as bagging (Bootstrap AGgregating). In general, combining different predictors together is a useful technique called **ensemble learning**.
- For decision trees though, doesn't make trees different enough from each other (e.g. if you have one really strong predictor, it'll always be the first split).

Idea 2: Only use a subset of features

- At each split, take a random subset of $ m $ features instead of choosing from all $ d $ of them.
- Rule of thumb: $ m \approx \sqrt d $ seems to work well.
- Key idea: For ensemble learning, you want the individual predictors to have low bias, high variance, and be uncorrelated with each other. That way, when you average them together, you have low bias AND low variance.
- **Random forest algorithm:** Fit $ T $ trees by using bagging and a random subset of features at each split. Predict by taking a vote from the $ T $ trees.

Example

In [44]:
```
# Let's use more features for prediction
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = (
    train_test_split(diabetes.drop(columns=['Outcome']), diabetes['Outcome']
)
```

In [45]:
```
from sklearn.ensemble import RandomForestClassifier

clf = RandomForestClassifier()
clf.fit(X_train, y_train)
clf.score(X_train, y_train)
```

Out[45]: 1.0

In [46]:
```
clf.score(X_test, y_test)
```

Out[46]: 0.8020833333333334

Compared to our previous best decision tree with depth 4:

In [47]:
```
dt = DecisionTreeClassifier(max_depth=4, criterion='entropy')
dt.fit(X_train, y_train)
dt.score(X_train, y_train)
```

Out[47]: 0.7829861111111112

In [48]:
```
dt.score(X_test, y_test)
```

Out[48]: 0.734375

Example: Modeling using text features

Example: Fake news

We have a dataset containing news articles and labels for whether the article was deemed "fake" or "real". Credit to https://github.com/KaiDMML/FakeNewsNet.

In [49]:
```
news = pd.read_csv('data/fake_news_training.csv')
news
```

Out[49]:

	baseurl	content	label
0	twitter.com	\njavascript is not available.\n\nwe've detect...	real
1	whitehouse.gov	remarks by the president at campaign event -- ...	real
2	web.archive.org	the committee on energy and commerce\nbarton: ...	real
...	...	...	...
658	politico.com	full text: jeff flake on trump speech transcri...	fake
659	pol.moveon.org	moveon.org political action: 10 things to know...	real
660	uspostman.com	uspostman.com is for sale\nyes, you can transf...	fake

661 rows × 3 columns

Goal: Use an article's content to predict its label.

In [50]:
```
news['label'].value_counts(normalize=True)
```

Out[50]:
```
label
real    0.55
fake    0.45
Name: proportion, dtype: float64
```

Question: What is the worst possible accuracy we should expect from a classifier, given the above distribution?

Aside: `CountVectorizer`

Entries in the `'content'` column are not currently quantitative! We can use the bag of words encoding to create quantitative features out of each `'content'`.

Instead of performing a bag of words encoding manually as we did before, we can rely on `sklearn`'s `CountVectorizer`. (There is also a `TfidfVectorizer`.)

In [51]:
```
from sklearn.feature_extraction.text import CountVectorizer
```

In [52]:
```
example_corp = ['hey hey hey my name is billy',
                'hey billy how is your dog billy']
```

In [53]:
```
count_vec = CountVectorizer()
count_vec.fit(example_corp)
```

Out[53]:
```
CountVectorizer()
```

Loading [MathJax]/extensions/Safe.js

`count_vec` learned a **vocabulary** from the corpus we `fit` it on.

In [54]:
```
count_vec.vocabulary_
```

Out[54]:
```
{'hey': 2,
 'my': 5,
 'name': 6,
 'is': 4,
 'billy': 0,
 'how': 3,
 'your': 7,
 'dog': 1}
```

In [55]:
```
count_vec.transform(example_corp).toarray()
```

Out[55]:
```
array([[1, 0, 3, 0, 1, 1, 1, 0],
       [2, 1, 1, 1, 1, 0, 0, 1]])
```

Note that the values in `count_vec.vocabulary_` correspond to the positions of the columns in `count_vec.transform(example_corp).toarray()`, i.e. `'billy'` is the first column and `'your'` is the last column.

In [56]:
```
example_corp
```

Out[56]:
```
['hey hey hey my name is billy', 'hey billy how is your dog billy']
```

In [57]:
```
pd.DataFrame(count_vec.transform(example_corp).toarray(),
             columns=pd.Series(count_vec.vocabulary_).sort_values().index)
```

Out[57]:

	billy	dog	hey	how	is	my	name	your
0	1	0	3	0	1	1	1	0
1	2	1	1	1	1	0	0	1

Creating an initial `Pipeline`

Let's build a `Pipeline` that takes in summaries and overall ratings and:

- Uses `CountVectorizer` to quantitatively encode summaries.
- Fits a `RandomForestClassifier` to the data.

But first, a train-test split (like **always**).

In [58]:
```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
```

In [59]:
```
X = news['content']
y = news['label']
```

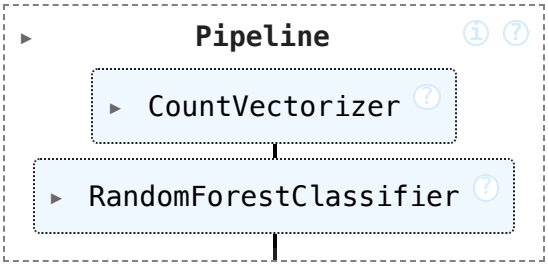
```
X_train, X_test, y_train, y_test = train_test_split(X, y)
```

To start, we'll create a random forest with 100 trees (`n_estimators`) each of which has a maximum depth of 3 (`max_depth`).

In [60]:
```
pl = Pipeline([
    ('cv', CountVectorizer()),
    ('clf', RandomForestClassifier(
        max_depth=3,
        n_estimators=100, # Uses 100 separate decision trees!
        random_state=42,
    ))
])
```

In [61]:
```
pl.fit(X_train, y_train)
```

Out[61]:



In [62]:
```
# Training accuracy.
pl.score(X_train, y_train)
```

Out[62]:
```
0.7656565656565657
```

In [63]:
```
# Testing accuracy.
pl.score(X_test, y_test)
```

Out[63]:
```
0.7228915662650602
```

The accuracy of our random forest is just under 73%, on the test set. How much better does it do compared to a classifier that predicts "real" every time?

In [64]:
```
y_train.value_counts(normalize=True)
```

Out[64]:
```
label
real    0.55
fake    0.45
Name: proportion, dtype: float64
```

In [65]:
```
# Distribution of predicted ys in the training set:

# stops scientific notation for pandas
pd.set_option('display.float_format', '{:.3f}'.format)
pd.Series(pl.predict(X_train)).value_counts(normalize=True)
```

Out[65]:
```
fake   0.655
real   0.345
Name: proportion, dtype: float64
```

In [66]:
```
len(pl.named_steps['cv'].vocabulary_) # Lots of features!
```

Out[66]:
```
26418
```

Choosing tree depth via `GridSearchCV`

We arbitrarily chose `max_depth=3` before, but it seems like that isn't working well. Let's perform a grid search to find the `max_depth` with the best generalization performance.

In [67]:
```
# Note that we've used the key clf__max_depth, not max_depth
# because max_depth is a hyperparameter of clf, not of pl.

hyperparameters = {
    'clf__max_depth': np.arange(2, 200, 20)
}
```

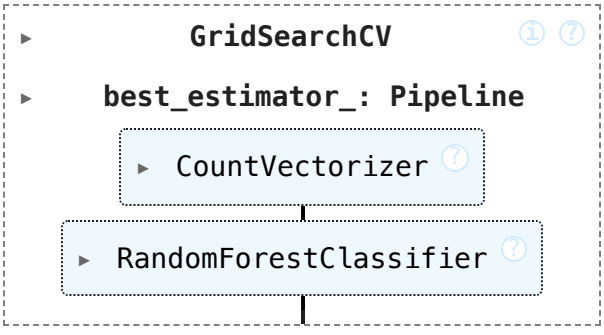
Note that while `pl` has already been `fit`, we can still give it to `GridSearchCV`, which will repeatedly re-`fit` it during cross-validation.

In [68]:
```
%%time

# Takes a few seconds to run — how many trees are being trained?
from sklearn.model_selection import GridSearchCV
grids = GridSearchCV(
    pl,
    n_jobs=-1, # Use multiple processors to parallelize
    param_grid=hyperparameters,
    return_train_score=True
)
grids.fit(X_train, y_train)
```

```
CPU times: user 1.04 s, sys: 158 ms, total: 1.2 s
Wall time: 7.57 s
```

Out[68]:



In [69]:
```
grids.best_params_
```

Loading [MathJax]/extensions/Safe.js

```
_depth': np.int64(42)}
```

Recall, `fit` `GridSearchCV` objects are estimators on their own as well. This means we can compute the training and testing accuracies of the "best" random forest directly:

In [70]:
```
# Training accuracy.
grids.score(X_train, y_train)
```

Out[70]:
```
0.9959595959595959
```

In [71]:
```
# Testing accuracy.
grids.score(X_test, y_test)
```

Out[71]:
```
0.8493975903614458
```

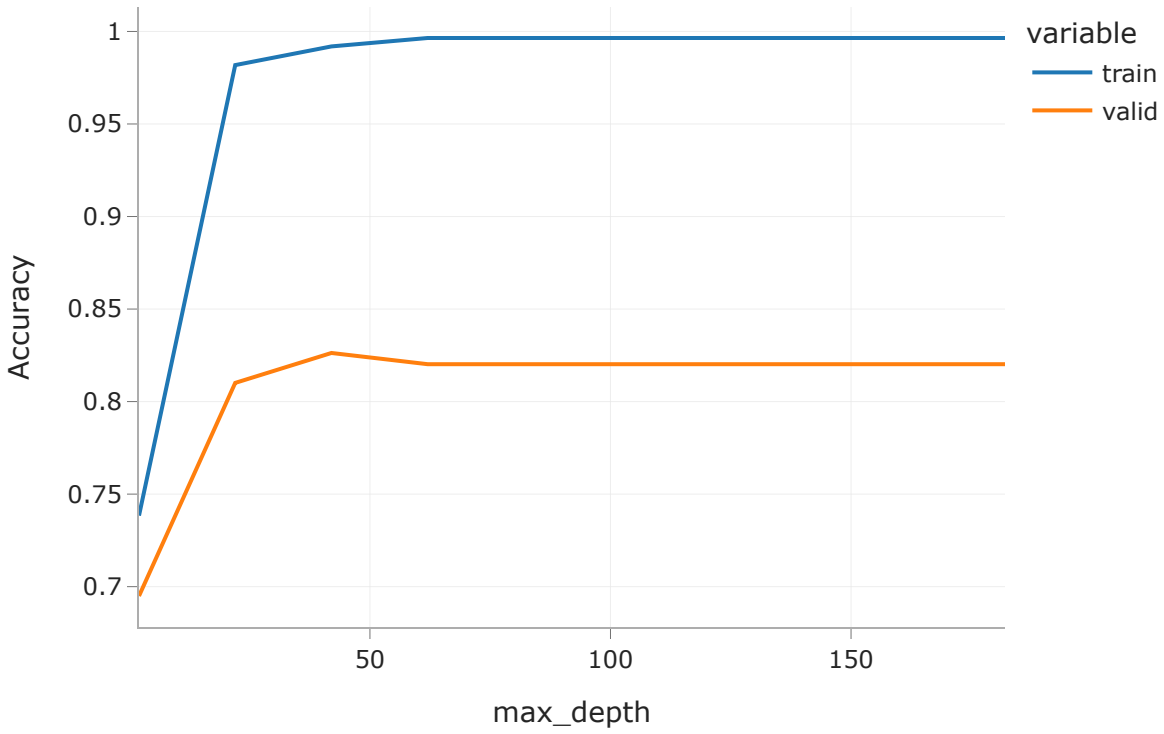
~10% better test set error!

Training and validation accuracy vs. depth

Below, we plot how training and validation accuracy varied with tree depth. Note that the y-axis here is accuracy, and that larger accuracies are better (unlike with RMSE, where smaller was better).

In [72]:
```
index = grids.param_grid['clf__max_depth']
train = grids.cv_results_['mean_train_score']
valid = grids.cv_results_['mean_test_score']
```

In [73]:
```
pd.DataFrame({'train': train, 'valid': valid}, index=index).plot().update_la
    xaxis_title='max_depth', yaxis_title='Accuracy'
)
```



Classifier Evaluation

Accuracy isn't everything!

$$ \text{accuracy} = \frac{\text{\# data points classified correctly}}{\text{\# data points}} $$

- Accuracy is defined as the proportion of predictions that are correct.
- It weighs all **correct** predictions the same, and weighs all **incorrect** predictions the same.
- But some incorrect predictions may be worse than others!
  - Example: Suppose you take a COVID test 🦠. Which is worse:
    - The test **saying you have COVID**, when you really don't, or
    - The test **saying you don't have COVID**, when you really do?

The Boy Who Cried Wolf 👦😭🐺

(source)

> A shepherd boy gets bored tending the town's flock. To have some fun, he cries out, "Wolf!" even though no wolf is in sight. The villagers run to

> protect the flock, but then get really mad when they realize the boy was playing a joke on them.

Repeat the previous paragraph many, many times.

> One night, the shepherd boy sees a real wolf approaching the flock and calls out, "Wolf!" The villagers refuse to be fooled again and stay in their houses. The hungry wolf turns the flock into lamb chops. The town goes hungry. Panic ensues.

The wolf classifier

- Predictor: Shepherd boy.
- Positive prediction: "There is a wolf."
- Negative prediction: "There is no wolf."

Some questions to think about:

- What is an example of an incorrect, positive prediction?
- Was there a correct, negative prediction?
- There are four possibilities. What are the consequences of each?
  - (predict yes, predict no) x (actually yes, actually no).

The wolf classifier

Below, we present a **confusion matrix**, which summarizes the four possible outcomes of the wolf classifier.

True Positive (TP): • Reality: A wolf threatened. • Shepherd said: "Wolf." • Outcome: Shepherd is a hero.	**False Positive (FP):** • Reality: No wolf threatened. • Shepherd said: "Wolf." • Outcome: Villagers are angry at shepherd for waking them up.
False Negative (FN): • Reality: A wolf threatened. • Shepherd said: "No wolf." • Outcome: The wolf ate all the sheep.	**True Negative (TN):** • Reality: No wolf threatened. • Shepherd said: "No wolf." • Outcome: Everyone is fine.

Outcomes in binary classification

ming **binary** classification, there are four possible outcomes.

(Note: A "positive prediction" is a prediction of 1, and a "negative prediction" is a prediction of 0.)

Outcome of Prediction	Definition	True Class
True positive (TP) ✅	The predictor **correctly** predicts the positive class.	P
False negative (FN) ❌	The predictor incorrectly predicts the negative class.	P
True negative (TN) ✅	The predictor **correctly** predicts the negative class.	N
False positive (FP) ❌	The predictor incorrectly predicts the positive class.	N

⬇️

	Predicted Negative	Predicted Positive
Actually Negative	TN ✅	FP ❌
Actually Positive	FN ❌	TP ✅

*The **confusion matrix** above is organized the same way that `sklearn`'s confusion matrices are (but differently than in the wolf example).*

Note that in the four acronyms – TP, FN, TN, FP – the **first letter** is whether the prediction is correct, and the **second letter** is what the prediction is.

Example: COVID testing 🦠

- UCSD Health administers hundreds of COVID tests a day. The tests are not fully accurate.
- Each test comes back either
  - positive, indicating that the individual has COVID, or
  - negative, indicating that the individual does not have COVID.
- **Question:** What is a TP in this scenario? FP? TN? FN?
- **TP:** The test predicted that the individual has COVID, and they do ✅.
- **FP:** The test predicted that the individual has COVID, but they don't ❌.
- **TN:** The test predicted that the individual doesn't have COVID, and they don't ✅.
- **FN:** The test predicted that the individual doesn't have COVID, but they do ❌.

Accuracy of COVID tests

The results of 100 UCSD Health COVID tests are given below.

	Predicted Negative	Predicted Positive
Actually Negative	TN = 90 ✅	FP = 1 ❌
Actually Positive	FN = 8 ❌	TP = 1 ✅

UCSD Health test results

🤔 **Question:** What is the accuracy of the test?

🙋 **Answer:** $$\text{accuracy} = \frac{TP + TN}{TP + FP + FN + TN} = \frac{1 + 90}{100} = 0.91$$

- **Followup:** At first, the test seems good. But, suppose we build a classifier that predicts that **nobody has COVID**. What would its accuracy be?
- **Answer to followup:** Also 0.91! There is severe **class imbalance** in the dataset, meaning that most of the data points are in the same class (no COVID). Accuracy doesn't tell the full story.

Recall

	Predicted Negative	Predicted Positive
Actually Negative	TN = 90 ✅	FP = 1 ❌
Actually Positive	FN = 8 ❌	TP = 1 ✅

UCSD Health test results

🤔 **Question:** What proportion of individuals who actually have COVID did the test **identify**?

🙋 **Answer:** $\frac{1}{1 + 8} = \frac{1}{9} \approx 0.11$

More generally, the **recall** of a binary classifier is the proportion of **actually positive instances** that are correctly classified. We'd like this number to be as close to 1 (100%) as possible.

$$\text{recall} = \frac{TP}{\text{\# actually positive}} = \frac{TP}{TP + FN}$$
To compute recall, look at the **bottom (positive) row** of the above confusion matrix.

Recall isn't everything, either!

$$\text{recall} = \frac{TP}{TP + FN}$$

🤔 **Question:** Can you design a "COVID test" with perfect recall?

🤷 **Answer:** Yes – **just predict that everyone has COVID!**

	Predicted Negative	Predicted Positive
Actually Negative	TN = 0 ✅	FP = 91 ❌
Actually Positive	FN = 0 ❌	TP = 9 ✅

everyone-has-COVID classifier

$$\text{recall} = \frac{TP}{TP + FN} = \frac{9}{9 + 0} = 1$$

Like accuracy, recall on its own is not a perfect metric. Even though the classifier we just created has perfect recall, it has 91 false positives!

Precision

	Predicted Negative	Predicted Positive
Actually Negative	TN = 0 ✅	FP = 91 ❌
Actually Positive	FN = 0 ❌	TP = 9 ✅

everyone-has-COVID classifier

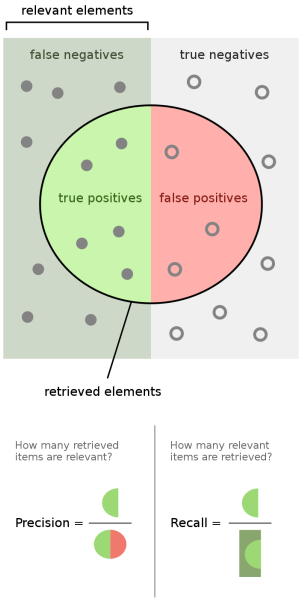
The **precision** of a binary classifier is the proportion of **predicted positive instances** that are correctly classified. We'd like this number to be as close to 1 (100%) as possible.

$$\text{precision} = \frac{TP}{\text{\# predicted positive}} = \frac{TP}{TP + FP}$$

To compute precision, look at the **right (positive) column** of the above confusion matrix.

- **Tip:** A good way to remember the difference between precision and recall is that in the denominator for 🅿️recision, both terms have 🅿️ in them (TP and FP).
- Note that the "everyone-has-COVID" classifier has perfect recall, but a precision of $\frac{9}{9 + 91} = 0.09$, which is quite low.
- 🚨 **Key idea:** There is a "tradeoff" between precision and recall. Ideally, you want both to be high. For a particular prediction task, one may be important than the other.

Precision and recall



(source)

Precision and recall

$$\text{precision} = \frac{TP}{TP + FP} \: \: \: \: \: \: \: \: \text{recall} = \frac{TP}{TP + FN}$$

Question 🤔

🤔 When might high **precision** be more important than high recall?

🤔 When might high **recall** be more important than high precision?

Question 🤔

Consider the confusion matrix shown below.

	Predicted Negative	Predicted Positive
Actually Negative	TN = 22 ✅	FP = 2 ❌
Actually Positive	FN = 23 ❌	TP = 18 ✅

What is the accuracy of the above classifier? The precision? The recall?

Summary, next time

Summary

- Decision trees, while interpretable, are prone to having high variance. There are several ways to control the variance of a decision tree:
  - Limit `max_depth` or increase `min_samples_split`.
  - Create a random forest, which is an ensemble of multiple decision trees, each fit to a different random resample of the training data, using a random sample of features.
- In order to tune model hyperparameters – that is, to find the hyperparameters that likely maximize performance on unseen data – use `GridSearchCV`.
- Accuracy alone is not always a meaningful representation of a classifier's quality, particularly when the classes are imbalanced.
  - Precision and recall are classifier evaluation metrics that consider the types of errors being made.
  - There is a "tradeoff" between precision and recall. One may be more important than the other, depending on the task.